PANEURÓPSKA VYSOKÁ ŠKOLA
FAKULTA INFORMATIKY

**FI-123456-12345**

# Názov bakalárskej práce

**Bakalárska práca**

**2026**

**Martin Muha**

**PANEURÓPSKA VYSOKÁ ŠKOLA**
**FAKULTA INFORMATIKY**

# Názov bakalárskej práce

Bakalárska práca

**Martin Muha**

| | |
|---|---|
| Študijný program: | Aplikovaná informatika |
| Študijný odbor: | Informatika |
| Školiace pracovisko: | Ústav aplikovanej informatiky |
| Školiteľ: | Mgr. Michal Drobný |

**Bratislava 2026**

PANEURÓPSKA VYSOKÁ ŠKOLA

Fakulta informatiky

# ZADANIE PRÁCE

Meno a priezvisko študenta:
Evidenčné číslo práce:
Študijný odbor:
Študijný program:
Forma a metóda štúdia:
Vedúci práce:
Ústav/katedra:
Dátum zadania práce:

Názov:

Anotácia: Cieľom práce je vytvorenie komplexného systému pre navigáciu v interiéroch s napojením na rozličné webové zdroje s možnosťou ich editovania a návrh shaderov pre ich zobrazenie. Súčasťou práce bude analýza použitia v reálnom nasadení vrátane implementácie a zobrazovania na rozličných dotykových zariadeniach.

VZOR

.....
RNDr. Ján Lacko, PhD.
vedúci práce

.....
Ing. Juraj Štefanovič, PhD.
vedúci ústavu

**Poďakovanie:**

Pri vypracovaní tejto bakalárskej práce by som sa rád poďakoval Mgr. Michalovi Drobnému za jeho odborné vedenie, cenné rady a konzultácie, ktoré mi významne pomohli pri spracovaní danej témy.

**Čestné prehlásenie:**

Čestne vyhlasujem, že záverečnú prácu som vypracoval samostatne a že som uviedol všetku použitú literatúru.

. . . . . .

Martin Muha

**Abstrakt**

Táto práca sa zameriava na návrh a analýzu webovej aplikácie RentMe, ktorá predstavuje inovatívnu digitálnu platformu pre prenájom vecí a poskytovanie služieb v rámci jedného integrovaného systému. Aplikácia kombinuje princípy zdieľanej ekonomiky a gig ekonomiky, pričom umožňuje používateľom nielen prenajímať a vyhľadávať rôzne druhy majetku, ale zároveň ponúkať a získavať pracovné príležitosti a služby prostredníctvom inzertného systému. Cieľom aplikácie RentMe je zjednodušiť proces sprostredkovania prenájmov a služieb, zvýšiť dostupnosť lokálnych pracovných príležitostí a podporiť efektívne využívanie existujúcich zdrojov. Práca sa venuje analýze konkurenčných platforiem, návrhu funkcionalít aplikácie, technologickej architektúre a identifikácii hlavných prínosov navrhovaného riešenia. Výsledkom je koncept univerzálnej webovej platformy, ktorá prepája viacero trhových segmentov do jedného ekosystému a reaguje na aktuálne trendy digitalizácie, flexibility práce a zdieľania zdrojov.

**Kľúčové slová:** webová aplikácia, prenájom, zdieľaná ekonomika, gig ekonomika, digitálna platforma

**Abstract**

This thesis focuses on the design and analysis of RentMe, a web application that represents an innovative digital platform for item rental and service provision within a single integrated system. The application combines principles of the sharing economy and gig economy, enabling users not only to rent and search for various types of assets, but also to offer and obtain employment opportunities and services through a classified advertisement system. The objective of the RentMe application is to streamline the process of mediating rentals and services, enhance the availability of local employment opportunities, and promote efficient utilization of existing resources. This work addresses the analysis of competing platforms, the design of application functionalities, technological architecture, and the identification of the main benefits of the proposed solution. The result is a concept of a universal web platform that interconnects multiple market segments into one ecosystem and responds to current trends in digitalization, work flexibility, and resource sharing.

**Keywords:** web application, rental, sharing economy, gig economy, digital platform

# Obsah


**1 Úvod** **9**

1.1 Motivácia a význam inzertných platforiem v súčasnosti . . . . . 9

1.1.1 Digitalizácia, dostupnosť a sieťový efekt . . . . . 9

1.1.2 Lokalizácia a geografický kontext . . . . . 9

1.1.3 Jednoduchosť a používateľská prívetivosť . . . . . 9

1.1.4 Dôvera, bezpečnosť a transparentnosť . . . . . 9

1.2 Definícia problému (centralizácia služieb a prenájmu) . . . . . 10

1.3 Ciele bakalárskej práce . . . . . 10

**2 Súčasný stav riešenej problematiky** **11**

2.1 Analýza konkurenčných platforiem a systémov . . . . . 11

2.1.1 Booking.com . . . . . 11

2.1.2 Fiverr.com . . . . . 12

2.1.3 Turo.com . . . . . 12

2.2 Ekonomické modely modernej inzercie . . . . . 13

2.2.1 Fenomén Micro-gig economy . . . . . 13

2.2.2 Model Peer-to-Peer (P2P) a zdieľaná ekonomika . . . . . 13

2.3 Technologické trendy a pridaná hodnota riešenia . . . . . 14

**3 Analýza a návrh systému** **14**

3.1 Funkčné požiadavky . . . . . 14

3.1.1 Návštevník . . . . . 14

3.1.2 Používateľ . . . . . 16

3.1.3 Administrátor . . . . . 16

3.2 Nefunkčné požiadavky . . . . . 17

3.2.1 Bezpečnosť . . . . . 17

3.2.2 Škálovateľnosť . . . . . 18

3.2.3 Výkon . . . . . 18

3.3 Návrh architektúry . . . . . 19

3.3.1 Štruktúra monorepa . . . . . 19

3.3.2 Model klient–server . . . . . 20

3.3.3 Dátový tok . . . . . 20

3.4 Databázový model . . . . . 21

3.4.1 Hlavné entity systému a ich význam . . . . . 21

3.4.2 Entitno-relačný diagram (ERD) . . . . . 23

3.5 REST API a JWT . . . . . 23

3.5.1 Návrh endpointov a smerovania . . . . . 23

3.5.2 Autentifikačný mechanizmus (JWT) . . . . . 24

3.5.3 Validácia a transformácia údajov . . . . . 24
3.5.4 Spracovanie chýb (Error Handling) . . . . . 25

**4 Metodika a použité technológie 25**
4.1 Full-stack vývoj v jazyku TypeScript . . . . . 25
4.1.1 Typová bezpečnosť a eliminácia chýb . . . . . 25
4.1.2 Zdieľanie kódu v monorepe . . . . . 26
4.1.3 Lepšia čitateľnosť, dokumentácia a Developer Experience (DX) . 26
4.1.4 Moderné funkcie, dekorátory a kompatibilita . . . . . 27
4.2 Backend: Framework NestJS a modulárna architektúra . . . . . 29
4.3 Frontend: Next.js (App Router), React a Tailwind CSS . . . . . 29
4.4 Perzistencia dát: PostgreSQL a Prisma ORM . . . . . 29
4.5 Nástroje pre správu kódu a kontajnerizáciu (Git, Docker Compose) . . 29

**5 Implementácia systému 29**
5.1 Štruktúra monorepa a zdieľanie typov (npm workspaces, packages/shared) 29
5.2 Realizácia backendových modulov (Auth, Ads, Categories, Search) . . . 29
5.3 Vývoj klientskych aplikácií . . . . . 29
5.3.1 Verejná platforma (vyhľadávanie, mapa, blog) . . . . . 29
5.3.2 Používateľská zóna (správa inzerátov a profilu) . . . . . 29
5.3.3 Administračný panel (CMS, vizuálny editor GrapesJS, štatistiky) 29
5.4 Bezpečnostné aspekty: Hashovanie hesiel (bcrypt), validácia vstupov (class-validator) a ochrana účtov (banovanie) . . . . . 29
5.5 Systém dynamických filtrov a kategórií . . . . . 29

**6 Testovanie a výsledky 29**
6.1 Overenie funkčnosti API (Swagger dokumentácia) . . . . . 29
6.2 Analytika a monitoring (ClickEvent tracking a segmentácia používateľov) 29
6.3 Vyhodnotenie používateľského rozhrania a responzivity . . . . . 29

**7 Diskusia a prínosy práce 29**
7.1 Správa obsahu bez nutnosti redeployu (SiteMenu, SiteConfig) . . . . . 29
7.2 Možnosti ďalšieho rozšírenia (platobná brána, externé úložisko médií) . 29

**8 Záver 29**
8.1 Zhrnutie dosiahnutých výsledkov . . . . . 29
8.2 Naplnenie stanovených cieľov . . . . . 29

# Zoznam obrázkov

1 Diagram použitia (Use Case) – funkčné požiadavky a aktéri systému . . 15
2 Tok zápisu dát po validácii: kontrolér, aplikačná služba, Prisma ORM a databáza PostgreSQL . . . . . 21

# 1 Úvod

## 1.1 Motivácia a význam inzertných platforiem v súčasnosti

V súčasnej digitálnej ére prechádza spôsob, akým spotrebitelia vyhľadávajú služby a prenájmy, zásadnou transformáciou. Tradičné formy inzercie sú nahrádzané interaktívnymi webovými platformami, ktoré umožňujú okamžitý prístup k informáciám v reálnom čase. Hlavnou motiváciou pre vývoj modernej inzertnej platformy je potreba konsolidovať rozptýlenú ponuku do jedného transparentného a bezpečného ekosystému.

### 1.1.1 Digitalizácia, dostupnosť a sieťový efekt

Moderný používateľ očakáva vysokú mieru flexibility a rýchlosti pri hľadaní odborných služieb či prenájmu. Kľúčovým faktorom úspechu je budovanie mena aplikácie a zvyšovanie jej návštevnosti, čím vzniká takzvaný sieťový efekt – s rastúcim počtom inzerátov a používateľov sa zvyšuje celkový dopyt po ponukách a hodnota platformy pre celú komunitu. Digitalizácia týchto procesov odbúrava administratívne bariéry a umožňuje efektívnejšie párovanie dopytu s ponukou.

### 1.1.2 Lokalizácia a geografický kontext

Pri službách a prenájmoch zohráva geografická poloha kritickú úlohu. Implementácia pokročilých mapových rozhraní a lokalizačných služieb (v projekte riešené cez knižnicu `Leaflet`) umožňuje používateľom vizualizovať dostupnosť ponúk vo svojom bezprostrednom okolí. Tento prístup zvyšuje používateľský komfort a znižuje logistické náklady, pričom geografické vyhľadávanie sa stáva štandardom odlišujúcim profesionálne riešenia od jednoduchých textových zoznamov.

### 1.1.3 Jednoduchosť a používateľská prívetivosť

Okrem funkčnej vybavenosti je kľúčovým prvkom významu platformy jej jednoduchosť. Intuitívne používateľské rozhranie zabezpečuje, že aplikácia je prístupná širokému spektru osôb bez ohľadu na ich technické zručnosti. Práve nízka bariéra vstupu a rýchly proces publikácie inzerátu priamo podmieňujú ochotu používateľov opakovane sa do systému vracať.

### 1.1.4 Dôvera, bezpečnosť a transparentnosť

Jednou z najväčších výziev online inzercie je eliminácia podvodného správania. Význam platformy spočíva v schopnosti garantovať bezpečnosť prostredníctvom aktívnej moderácie, systémov na nahlasovanie podozrivých aktivít (modul `reports`) a prísnych

pravidiel pre schvaľovanie inzerátov administrátorom. Tieto prvky, spolu s prehľadnou históriou správ, budujú nevyhnutnú dôveru medzi subjektmi v digitálnom priestore.

## 1.2 Definícia problému (centralizácia služieb a prenájmu)

Súčasný digitálny trh so službami a prenájmom na Slovensku je príliš rozdrobený, čo komplikuje kontakt medzi tými, ktorí službu ponúkajú, a tými, ktorí ju hľadajú. Hlavným problémom je chýbajúca univerzálna platforma, ktorá by dokázala spojiť rôzne profesie a typy prenájmu na jednom mieste. Používatelia sú dnes nútení hľadať riešenia cez viaceré, často špecializované alebo neformálne kanály, ktoré však nespĺňajú všetky ich potreby.

Dobrým príkladom špecializácie je portál `pretlak.com`. Ponúka síce moderné prostredie, ale zameriava sa takmer výhradne na IT, marketing a kreatívny priemysel. Podobne sú na tom aj skupiny na sociálnych sieťach, napríklad skupina `Programátori`, ktorá slúži len úzkemu okruhu odborníkov na vývoj. Ostatné remeslá a profesie tak často končia na portáli `bazos.sk`. Ten je však v prvom rade určený na predaj tovaru, nie na správu služieb. Navyše, jeho vizuálna stránka a ovládanie cielia skôr na staršie generácie, čo pre mladších používateľov, zvyknutých na moderné a rýchle aplikácie, predstavuje technologickú prekážku.

Ďalšou výraznou slabinou je chýbajúca štruktúra a slabé možnosti filtrovania. Na sociálnych sieťach sa príspevky radia za sebou podľa času pridania, kvôli čomu zaujímavé ponuky rýchlo zapadnú. Keďže chýba podrobné vyhľadávanie, používateľ musí manuálne prechádzať množstvo inzerátov, ktoré ho nezaujímajú. Problémom je aj chýbajúci geografický prehľad. Bez mapy a presného určenia polohy je veľmi ťažké a zdĺhavé nájsť niekoho, kto poskytuje služby priamo v okolí používateľa.

Z pohľadu bezpečnosti sú sociálne siete a fóra rizikové, pretože im chýbajú nástroje na kontrolu obsahu alebo overovanie osôb. Ak niekto poruší pravidlá, administrátor nemá k dispozícii systém, ktorý by okamžite zasiahol, čo vytvára priestor pre podvody. Prevádzkovatelia týchto skupín navyše nevedia sledovať, o čo majú ľudia skutočný záujem, pretože nemajú prístup k analytickým dátam. Moja práca na tieto nedostatky reaguje vytvorením centralizovanej platformy, ktorá spája všetky potrebné funkcie do jedného prehľadného systému s jasnými pravidlami a jednoduchým ovládaním.

## 1.3 Ciele bakalárskej práce

Hlavným cieľom tejto bakalárskej práce je navrhnúť a vytvoriť modernú webovú platformu, ktorá slúži na uverejňovanie a správu inzerátov v oblasti služieb a prenájmu. Celý systém je navrhnutý tak, aby na jednom mieste prepojil bežných používateľov, poskytovateľov služieb a administrátorov, čím nahrádza súčasné neprehľadné hľadanie na sociálnych sieťach alebo všeobecných inzertných weboch.

Pre dosiahnutie tohto hlavného cieľa som si stanovil niekoľko konkrétnych úloh:

Prvou úlohou je vytvorenie stabilného a bezpečného technického zázemia (backendu) pomocou frameworku `NestJS`. Tento systém musí zvládnuť bezpečnú registráciu a prihlasovanie používateľov cez `JWT` tokeny, správne ukladať dáta do databázy `PostgreSQL` a zabezpečiť, aby sa k citlivým údajom nedostal nikto nepovolaný. Dôležitým bodom je aj kontrola vstupných údajov, aby sa predišlo chybám alebo útokom na aplikáciu.

Druhou úlohou je vývoj troch samostatných webových aplikácií pomocou technológie `Next.js`, ktoré sú prispôsobené konkrétnym skupinám ľudí. Verejná časť umožní návštevníkom rýchlo vyhľadávať inzeráty aj pomocou interaktívnej mapy, používateľská zóna dovolí registrovaným členom spravovať svoje ponuky a správy, a administračný panel poskytne nástroje na schvaľovanie obsahu a sledovanie štatistík. Pri tvorbe vzhľadu sa zameriavam na jednoduchosť a moderný dizajn, aby bola aplikácia príjemná pre mladšiu generáciu, no zároveň zrozumiteľná pre každého.

Posledným dôležitým cieľom je implementácia analytických nástrojov a funkcií na moderovanie obsahu. Systém bude automaticky sledovať kliknutia na inzeráty, čo správcovi pomôže pochopiť, o ktoré kategórie je najväčší záujem. Zároveň bude platforma obsahovať funkcie na nahlasovanie nevhodných inzerátov a možnosť zablokovať problémových používateľov, čím sa zvýši celková bezpečnosť a dôveryhodnosť celého systému.

# 2 Súčasný stav riešenej problematiky

V tejto kapitole podrobujem analýze aktuálne dostupné riešenia v oblasti digitálnej inzercie a zdieľanej ekonomiky. Zameriavam sa na to, ako svetoví lídri a lokálne slovenské portály pristupujú k správe služieb a prenájmu, pričom identifikujem funkčné a technologické nedostatky, ktoré moja platforma odstraňuje.

## 2.1 Analýza konkurenčných platforiem a systémov

Pre pochopenie štandardov v odvetví je nevyhnutné analyzovať systémy, ktoré definujú súčasné trendy v oblasti digitálnych trhovísk (marketplace). Tieto platformy reprezentujú rôzne logické prístupy k spracovaniu dát a budovaniu dôvery medzi používateľmi.

### 2.1.1 Booking.com (Model komplexnej rezervácie a správy kapacít)

**Charakteristika a logika:** Booking.com operuje na báze pokročilých algoritmov pre správu voľných kapacít v reálnom čase. Systém je postavený na okamžitej potvrdenej rezervácii a dynamickej tvorbe cien, čo si vyžaduje robustnú architektúru na zamedzenie duplicitných rezervácií.

**Výhody:** Extrémne podrobné hierarchické filtrovanie a vysoká miera dôveryhodnosti budovaná cez overené recenzie.

**Nevýhody:** Systém je uzavretý výhradne pre segment ubytovania. Pre malých lokálnych poskytovateľov iných služieb sú prekážkou vysoké provízie a rigidita systému, ktorý neumožňuje inzerovať predmety dennej potreby alebo remeselné práce.

**Porovnanie s vlastným riešením:** Moja platforma preberá logiku podrobného filtrovania meta-údajov, avšak aplikuje ju univerzálne. Vďaka flexibilnej schéme v `Prisma ORM` umožňujem inzerátom priraďovať rôzne špecifikácie (napr. technické parametre náradia) bez nutnosti meniť základnú architektúru databázy.

### 2.1.2 Fiverr.com (Model produktizácie služieb – Micro-gig economy)

**Charakteristika a logika:** Fiverr transformoval nehmotné služby na jasne definované balíčky (produkty). Logika fungovania spočíva v eliminácii vyjednávania o cene a rozsahu práce, čím sa proces nákupu služby približuje klasickému elektronickému obchodu.

**Výhody:** Prehľadnosť nákupného procesu a automatizované filtrovanie kvality poskytovateľov cez systém úrovní.

**Nevýhody:** Orientácia takmer výhradne na digitálne produkty doručiteľné online. Absencia lokalizačnej vrstvy a máp znemožňuje efektívne sprostredkovanie fyzických služieb v konkrétnom meste.

**Porovnanie s vlastným riešením:** Implementujem logiku „služba ako produkt“, ale obohacujem ju o kľúčový geografický kontext. Integrácia mapových podkladov cez knižnicu `Leaflet` a ukladanie GPS súradníc v `PostgreSQL` umožňuje používateľom nájsť odborníka v ich bezprostrednom okolí.

### 2.1.3 Turo.com (Model Peer-to-Peer zdieľania majetku)

**Charakteristika a logika:** Turo reprezentuje model zdieľanej ekonomiky v doprave, kde súkromné osoby monetizujú svoj nevyužitý majetok (automobily). Logika systému stojí na overovaní identity a synchronizácii s kalendárom vlastníka.

**Výhody:** Efektívne využitie existujúcich zdrojov a prepracovaná vizuálna prezentácia majetku.

**Nevýhody:** Ide o jednoúčelovú aplikáciu. Ak používateľ hľadá iný typ prenájmu (napr. stavebnú techniku), je nútený využiť ďalší samostatný systém.

**Porovnanie s vlastným riešením:** Navrhované riešenie predstavuje integrovaný ekosystém, ktorý konsoliduje fragmentované potreby lokálneho trhu do jedinej platformy, čím odstraňuje nutnosť využívania viacerých jednoúčelových aplikácií.

## 2.2 Ekonomické modely modernej inzercie

Pre pochopenie fungovania navrhovanej platformy je nevyhnutné definovať dva kľúčové ekonomické koncepty, ktoré v poslednom desaťročí transformovali globálny trh práce a služieb.

### 2.2.1 Fenomén Micro-gig economy

Micro-gig economy (často označovaná aj ako „platformová ekonomika“) predstavuje model, v ktorom sú dlhodobé pracovné pomery nahrádzané krátkodobými, presne definovanými úlohami, takzvanými „gigmi“. Tento model zmenil vnímanie práce z fixného zamestnania na poskytovanie konkrétnej hodnoty v reálnom čase.

**Logika a fungovanie:** Základom je digitalizácia dopytu. Platforma funguje ako algoritmus, ktorý páruje voľné kapacity dodávateľa s okamžitou potrebou odberateľa. Na rozdiel od klasickej inzercie sa tu hľadá „riešenie problému“. Príkladom je platforma Fiverr, kde je služba predávaná ako vopred zabalený produkt s fixnou cenou a termínom dodania.

**Výhody pre trh:** Tento model umožňuje obrovskú mieru flexibility. Freelanceri môžu pracovať pre viacerých klientov súčasne a záujemcovia majú prístup k expertom bez nutnosti uzatvárania zložitých zmlúv.

**Aplikácia v mojom projekte:** Moja platforma preberá z tohto modelu princíp „produktizácie“ služieb. Vytvoril som systém, kde si používateľ nevyberá len osobu, ale konkrétnu ponuku s jasnými parametrami. Vďaka modulu `Advertisements` a dynamickým filtrom umožňujem inzerentom presne definovať rozsah svojej „mikro-zákazky“, čím znižujem čas potrebný na vyjednávanie.

### 2.2.2 Model Peer-to-Peer (P2P) a zdieľaná ekonomika

Model Peer-to-Peer (rovný s rovným) je základným stavebným kameňom zdieľanej ekonomiky. Ide o transakcie, ktoré prebiehajú priamo medzi dvoma koncovými používateľmi, pričom platforma vystupuje len ako technologický sprostredkovateľ a garant bezpečnosti.

**Logika zdieľania majetku:** Hlavnou myšlienkou je efektívna monetizácia nevyužitých statkov. Tento prístup je nielen ekonomicky výhodný pre obe strany, ale aj ekologicky udržateľný. Platformy ako Airbnb dokázali, že technológia dokáže prekonať prirodzenú nedôveru medzi cudzími ľuďmi.

**Budovanie dôvery:** Kľúčovým prvkom P2P logiky je reputačný systém a moderácia. Bez možnosti nahlásiť nekorektné správanie by tento model skolaboval.

**Aplikácia v mojom projekte:** Moja platforma je postavená na P2P princípe najmä v segmente prenájmu. Implementoval som moduly `Messages` pre bezpečnú

komunikáciu a `Reports` pre nahlasovanie incidentov, čo sú nevyhnutné nástroje na udržanie poriadku v P2P komunite. Na rozdiel od platforiem ako Bazoš, môj systém vďaka administrátorskému panelu umožňuje aktívne zasahovať a chrániť používateľov.

## 2.3 Technologické trendy a pridaná hodnota riešenia

Aby aplikácia obstála v konkurencii, využíva moderný technologický stack, ktorý rieši slabiny starších portálov.

**Server-side Rendering (`Next.js`):** Zabezpečuje rýchle načítanie a lepšiu indexáciu inzerátov vyhľadávačmi (SEO), čo je kľúčové pre návštevnosť verejnej platformy.

**Typová bezpečnosť (`TypeScript`):** Použitie monorepa umožňuje zdieľať dátové modely (DTO) medzi API a frontend aplikáciami, čím sa minimalizuje riziko nekonzistencií v dátach.

**Analytika (`ClickEvent`):** Na rozdiel od neformálnych riešení (Facebook skupiny) systém monitoruje správanie používateľov (kliknutia, segmentácia podľa pohlavia či typu účtu), čo umožňuje administrátorovi optimalizovať kategórie podľa skutočného dopytu.

# 3 Analýza a návrh systému

Návrh systému vychádza z rozdelenia platformy na štyri základné časti: verejná platforma, používateľské konto, admin panel a backend API. Tento prístup, realizovaný pomocou monorepo štruktúry a oddelených `Next.js` aplikácií, umožňuje presne definovať oprávnenia pre rôzne skupiny používateľov a izolovať ich používateľské rozhrania.

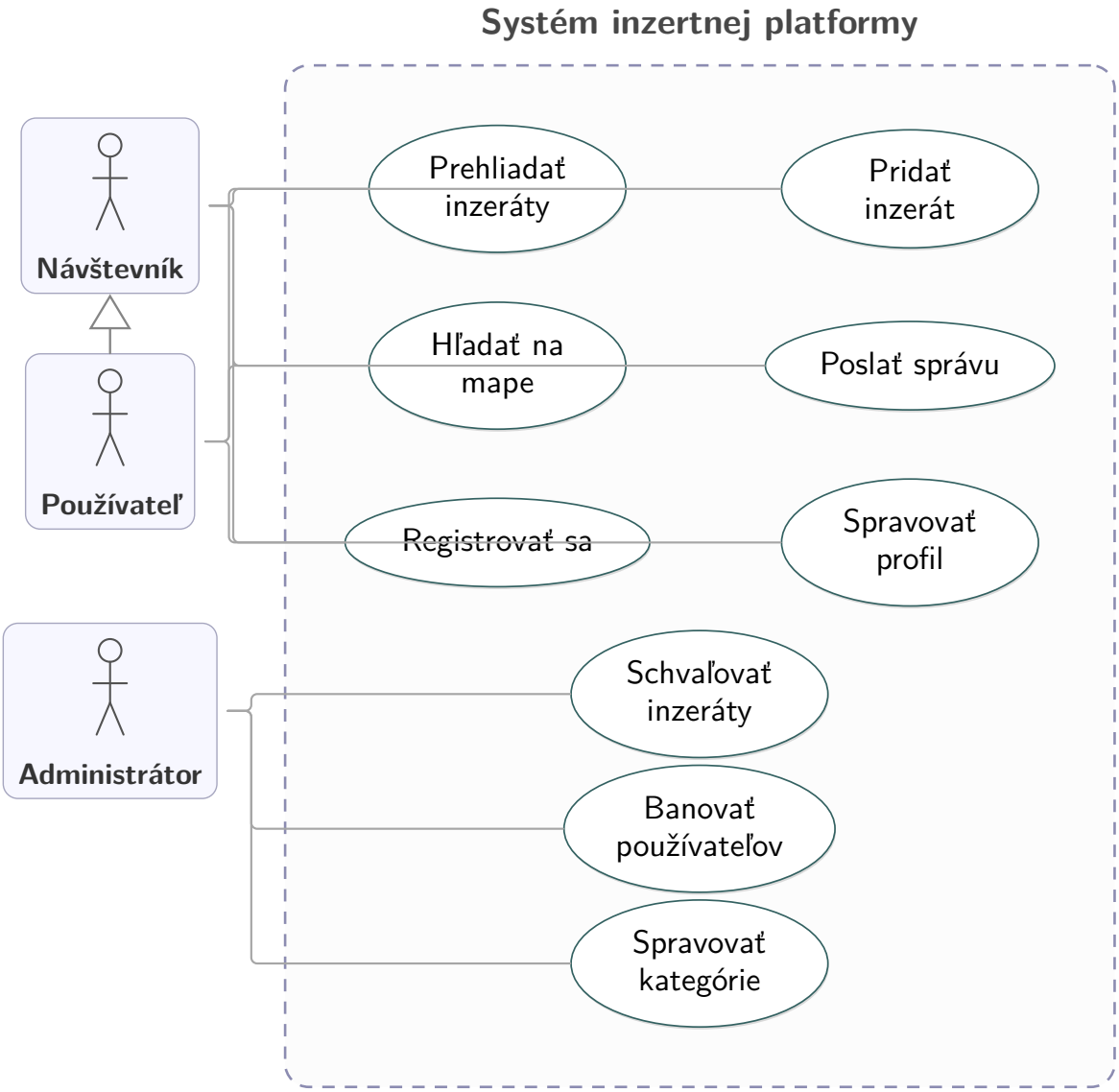
## 3.1 Funkčné požiadavky

Funkčné požiadavky sú rozdelené podľa rolí, ktoré v systéme vystupujú. Každá rola má priradený špecifický rozsah akcií, ktoré môže v rámci aplikácie vykonávať. Na úrovni databázy je rola používateľa definovaná enumerátorom `UserRole` (s hodnotami `ADMIN` a `USER`).

Obrázok 1 zobrazuje diagram použitia (Use Case): vzťahy medzi aktérmi (Návštevník, Používateľ, Administrátor) a vybranými prípadmi použitia. Pri aktérovi Používateľ je šípkou zovšeobecnenia (UML generalization) znázornené, že okrem vlastných prípadov použitia zdedí aj možnosti návštevníka.

### 3.1.1 Rola: Návštevník (neprihlásený používateľ)

Návštevník je anonymný používateľ, ktorý primárne konzumuje verejne dostupný obsah bez potreby registrácie.




Obr. 1: Diagram použitia (Use Case) – funkčné požiadavky a aktéri systému

- **Prehliadanie obsahu:** Používateľ môže prehliadať kategórie, zoznamy inzerátov a detaily jednotlivých ponúk.
- **Vyhľadávanie a filtrácia:** Možnosť vyhľadávať inzeráty podľa kľúčových slov a filtrovať výsledky na základe dynamických parametrov kategórie.
- **Mapové zobrazenie:** Prístup k interaktívnej mape, ktorá vizualizuje polohu inzerovaných služieb a prenájmov.
- **Čítanie blogu:** Možnosť čítať publikované články a statické informačné stránky (napr. podmienky používania).
- **Registrácia:** Návštevník má možnosť vytvoriť si používateľský účet.

### 3.1.2 Rola: Používateľ (registrovaný člen)

Používateľ má po prihlásení prístup k funkciám, ktoré umožňujú aktívnu participáciu na trhu služieb. Prístup k týmto funkciám je na backende chránený pomocou JWT autentifikácie (v `NestJS` implementovanej cez `JwtAuthGuard`).

- **Správa profilu:** Úprava osobných údajov, firemných údajov (napr. IČO, DIČ) a zmena prístupového hesla.
- **Manažment inzerátov:** Tvorba nových inzerátov (služba alebo prenájom), ich úprava, mazanie a sledovanie stavu schválenia. V databáze môže inzerát nadobúdať stavy definované enumerátorom `AdvertisementStatus` (`PENDING`, `DRAFT`, `ACTIVE`, `INACTIVE`, `ARCHIVED`).
- **Komunikačný modul:** Odosielanie a prijímanie správ (dopytov) k inzerátom prostredníctvom integrovaného chatu.
- **Obľúbené položky:** Ukladanie zaujímavých inzerátov do vlastného zoznamu pre neskorší návrat (entita `Favorite`).
- **Nahlasovanie (Reportovanie):** Možnosť nahlásiť inzerát, ktorý porušuje pravidlá platformy.

### 3.1.3 Rola: Administrátor

Administrátor má najvyššiu úroveň oprávnení a dohliada na technický a obsahový chod platformy cez dedikovaný admin panel. Na úrovni API sú tieto endpointy chránené kombináciou `JwtAuthGuard` a vlastného `RolesGuard`, ktorý overuje, či má prihlásený používateľ rolu `ADMIN`.

```
@Controller('admin')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(UserRole.ADMIN)
export class AdminController {
  // ...
  @Get('stats')
  getStats() {
    return this.adminService.getStats();
  }
}
```

Výpis 1: Ukážka zabezpečenia administrátorského endpointu v NestJS.

- **Moderácia obsahu:** Schvaľovanie, zamietanie alebo archivácia inzerátov a riešenie prijatých nahlásení od používateľov.
- **Správa používateľov:** Prehľad o registrovaných členoch s možnosťou udelenia dočasného alebo trvalého banu (blokovania účtu). Databázový model `User` obsahuje na tento účel polia `banned`, `bannedUntil` a `banReason`.
- **CMS (Content Management System):** Tvorba a editácia statických stránok, blogových príspevkov a správa položiek v navigačnom menu.
- **Konfigurácia systému:** Nastavovanie kategórií, hierarchie filtrov a globálnych parametrov platformy.
- **Analytika a prehľady:** Sledovanie štatistík návštevnosti, kliknutí (`ClickEvent`) a trendov v inzercii prostredníctvom grafov.

## 3.2 Nefunkčné požiadavky

Nefunkčné požiadavky definujú technické limity a prevádzkové vlastnosti, ktoré sú nevyhnutné pre spoľahlivý chod platformy. Pri systéme, ktorý sprostredkováva interakciu medzi cudzími ľuďmi a narába s ich citlivými údajmi, som kládol dôraz predovšetkým na bezpečnosť, jednoduchú rozšíriteľnosť a rýchlu odozvu používateľského rozhrania.

### 3.2.1 Bezpečnosť a ochrana údajov

Vzhľadom na to, že platforma spracováva používateľské profily a súkromnú komunikáciu, je bezpečnosť prioritou.

**Autentifikácia a autorizácia:** Prístup k citlivým častiam API je chránený platným JSON Web Tokenom (JWT). V prostredí `NestJS` je táto vrstva realizovaná pomocou

stratégie `passport-jwt` a chránená globálnymi guardmi (`JwtAuthGuard`, `RolesGuard`), ktoré pri každej požiadavke overujú identitu aj priradenú rolu používateľa.

**Hashovanie hesiel:** Heslá nie sú v databáze `PostgreSQL` nikdy ukladané v čitateľnej podobe. Na ich bezpečnú transformáciu využívam kryptografický algoritmus `bcrypt` so soľou (salt), čo znemožňuje ich spätné dešifrovanie.

**Validácia vstupov:** Aby som predišiel útokom typu mass assignment alebo spracovaniu neúplných dát, systém automaticky filtruje každú prichádzajúcu požiadavku. Je to zabezpečené globálnym nastavením `ValidationPipe` (režimy `whitelist` a `forbidNonWhitelisted`), ktoré v spolupráci s knižnicou `class-validator` a DTO triedami prísne kontroluje štruktúru vstupov.

**CORS a blokovanie zneužitia:** API má presne definované povolené zdroje (origins), čím sa bráni neautorizovaným volaniam z cudzích domén. Zároveň systém obsahuje mechanizmus banovania, ktorý administrátorovi umožňuje okamžite zablokovať prístup nahláseným používateľom (využitím polí `banned` a `banReason` v databáze).

### 3.2.2 Škálovateľnosť a udržateľnosť systému

Platforma je navrhnutá tak, aby bolo možné v budúcnosti jednoducho pridávať nové funkcie bez nutnosti prepisovania existujúceho kódu.

**Architektúra monorepa:** Využitie `npm workspaces` mi umožnilo logicky oddeliť frontendové aplikácie (`platform`, `user`, `admin`) od backendu (`api`). Vďaka zdieľanému balíku `packages/shared` sú dátové typy a DTO konzistentné naprieč celým projektom, čo výrazne znižuje chybovosť pri vývoji.

**Modularita a ORM:** Backend je rozdelený do samostatných doménových modulov (napr. `AuthModule`, `Advertisements Module`), čo uľahčuje údržbu a testovanie. Použitie `Prisma ORM` zasa zabezpečuje, že zmeny v dátovom modeli sú typovo bezpečné a ľahko nasaditeľné pomocou verziovaných migrácií.

**Kontajnerizácia:** Celý projekt je pripravený na nasadenie cez Docker. Súbor `docker-compose.yml` definuje izolované prostredia pre databázu aj Node.js služby, čo zaručuje, že aplikácia sa bude správať identicky na mojom počítači aj na produkčnom serveri.

### 3.2.3 Výkon, odozva a SEO

Rýchlosť načítania a viditeľnosť vo vyhľadávačoch sú kľúčové pre úspech každého inzertného portálu.

**Server-side Rendering (SSR):** Vďaka `Next.js` (verzia 16+) sa obsah inzerátov generuje priamo na serveri. To zabezpečuje nielen okamžitú odozvu pre používateľa, ale

aj ideálnu indexáciu Google robotmi, keďže HTML dokument už pri načítaní obsahuje dôležité SEO metadáta.

**Optimalizácia databázy:** Pre zrýchlenie vyhľadávania pri veľkom množstve dát som v Prisma schéme definoval indexy (`@@index`) pre najčastejšie filtrované polia, ako sú `status`, `slug` či `categoryId`.

**Responzívny dizajn:** Používateľské rozhranie postavené na `Tailwind CSS` využíva prístup mobile-first. To zaručuje, že platforma je plne funkčná a vizuálne príťažlivá na smartfónoch aj veľkých monitoroch.

**Práca s médiami:** API je optimalizované na spracovanie väčších objemov dát (limit 50 MB), čo umožňuje bezproblémový prenos obrázkov inzerátov kódovaných vo formáte `base64`.

## 3.3 Návrh architektúry: Model klient–server v prostredí monorepa

Architektúra systému je postavená na modernom koncepte monorepa, ktorý umožňuje efektívnu správu viacerých samostatných aplikácií a zdieľaných knižníc v rámci jedného úložiska (repozitára). Celý ekosystém je rozdelený na klientske aplikácie (front-end) a centrálny server (back-end), ktoré medzi sebou komunikujú prostredníctvom protokolu HTTP a formátu JSON.

### 3.3.1 Štruktúra monorepa a zdieľanie kódu

Projekt využíva nástroj `npm workspaces`, ktorý zabezpečuje logické rozdelenie kódu do dvoch hlavných kategórií: aplikácie (`apps/`) a balíčky (`packages/`).

- **Aplikácie (`apps/`):** Obsahujú tri nezávislé `Next.js` aplikácie s App Routerom (verzia 16+): verejnú platformu (`apps/platform`), používateľské konto (`apps/user`) a administráciu (`apps/admin`). Štvrtou aplikáciou je centrálny `NestJS` API server (`apps/api`).
- **Zdieľané balíčky (`packages/`):** Kľúčovým prvkom je balík `packages/shared`, ktorý obsahuje spoločné TypeScript typy, enumerátory (napríklad `UserRole` a `AdvertisementStatus`) a objekty DTO (Data Transfer Objects). Vďaka tomu je zaručené, že ak sa zmení štruktúra dát na backende, zmena sa okamžite premietne aj do frontendu, čím sa predchádza nekonzistenciám a chybám pri kompilácii.
- **Databázový balík:** Balík `packages/database` obsahuje definíciu Prisma schémy (`schema.prisma`), vygenerovaného Prisma klienta a migračné skripty. Týmto je

prístup k databáze centralizovaný a zdieľaný pre potreby API.

### 3.3.2 Model klient–server

Systém funguje na princípe striktného oddelenia prezentácie od aplikačnej logiky a ukladania dát.

- **Backend API (Server):** Je postavený na frameworku `NestJS` a funguje ako centrálny bod systému. Zabezpečuje komunikáciu s databázou `PostgreSQL` cez `Prisma ORM`, vykonáva striktnú validáciu dát (pomocou `ValidationPipe` a `class-validator`) a overuje oprávnenia používateľov cez JWT tokeny (pomocou `Passport.js`). API zároveň poskytuje automaticky generovanú Swagger dokumentáciu (`/api/docs`).

- **Klientske aplikácie (Klienti):** Tri samostatné aplikácie v `Next.js` komunikujú s API prostredníctvom asynchrónnych HTTP požiadaviek (typicky smerovaných na `NEXT_PUBLIC_API_URL`). Tento prístup umožňuje, aby každá časť systému fungovala nezávisle a mala vlastné, optimalizované používateľské rozhranie prispôsobené konkrétnej role (napr. vizuálny editor `GrapesJS` v admin paneli vs. interaktívna mapa `Leaflet` na verejnej platforme).

### 3.3.3 Dátový tok a komunikácia

Keď používateľ vykoná akciu (napríklad pridá nový inzerát), proces prebieha v nasledujúcich krokoch:

1. **Iniciácia na klientovi:** Frontend (napr. `apps/user`) zhromaždí dáta z formulára a odošle HTTP `POST` požiadavku. Dáta sú už na strane klienta typované podľa zdieľaných DTO typov z balíka `packages/shared`. Do hlavičky `Authorization` je vložený JWT token prihláseného používateľa.

2. **Prijatie a validácia na serveri:** API server prijme požiadavku. Najprv overí identitu používateľa cez `JwtAuth Guard` (prípadne rolu cez `Roles Guard`). Následne globálna `Validation Pipe` skontroluje telo požiadavky voči DTO triede (odstráni neznáme polia a overí povinné parametre).

3. **Zápis do databázy:** Ak sú dáta validné, príslušný kontrolér (napr. `Advertisements Controller`) posunie požiadavku do aplikačnej služby (`Advertisements Service`). Tá pomocou `Prisma ORM` vykoná asynchrónny zápis do databázy `PostgreSQL`.

4. **Odpoveď servera:** Server vráti odpoveď v JSON formáte (napr. vytvorený objekt inzerátu s vygenerovaným UUID a HTTP status kódom `201 Created`).
5. **Aktualizácia používateľského rozhrania:** Frontend prijme odpoveď a zareaguje aktualizáciou rozhrania (napr. presmerovaním používateľa na detail novovytvoreného inzerátu alebo zobrazením notifikácie o úspechu).

Obrázok 2 znázorňuje zjednodušený tok vrstiev pri zápise po úspešnej validácii (krok 3), v súlade s architektúrou NestJS a perzistenciou cez Prisma.

Obr. 2: Tok zápisu dát po validácii: kontrolér, aplikačná služba, Prisma ORM a databáza PostgreSQL

## 3.4 Databázový model a entitno-relačný diagram (ERD)

Pre potreby perzistencie dát využívam relačnú databázu `PostgreSQL`, ktorá je v rámci vývoja spravovaná pomocou nástroja `Prisma ORM`. Tento prístup mi umožňuje definovať dátový model v prehľadnom deklaratívnom formáte (`schema.prisma`) a následne automaticky generovať migračné skripty a typovo bezpečný klient pre backendovú aplikáciu.

### 3.4.1 Hlavné entity systému a ich význam

Logický model databázy pozostáva z niekoľkých kľúčových entít, ktoré sú navzájom prepojené reláciami typu 1:N (one-to-many) alebo M:N (many-to-many).

- **User (Používateľ):** Predstavuje identitu v systéme. Okrem základných údajov ako e-mail a hashované heslo obsahuje informácie o roli (`ADMIN` / `USER`), firemné údaje pre predajcov (IČO, DIČ, `isCompany`), predajný plán (`SellerPlan`) a polia súvisiace s moderáciou, ako sú `banned`, `bannedUntil` a `banReason`.
- **Advertisement (Inzerát):** Centrálna entita obsahujúca detaily o ponúkanej službe alebo prenájme (`AdvertisementType`). Inzerát je viazaný na konkrétneho autora (používateľa) a kategóriu. Obsahuje stavy definované enumerátorom `AdvertisementStatus` (`PENDING`, `DRAFT`, `ACTIVE`, `INACTIVE`, `ARCHIVED`), lokalitu (GPS súradnice `latitude`, `longitude`), cenu a špecifikácie uložené vo formáte JSON (`specifications`) pre maximálnu flexibilitu parametrov bez nutnosti meniť databázovú schému pri každom novom parametri.
- **Category (Kategória):** Zabezpečuje hierarchické usporiadanie inzerátov. Podporuje vnorenie (parent-child relácie cez `parentId`), obsahuje SEO polia (`metaTitle`, `metaDescription`, `slug`) a definuje poradie zobrazenia v navigácii (`order`).
- **Filter (Dynamický filter):** Úzko spolupracuje s kategóriami. Definuje, aké dodatočné parametre (napr. typ `TEXT`, `NUMBER`, `SELECT`, `MULTISELECT`, definované v `FilterType`) môže používateľ pri inzeráte v danej kategórii zadať a podľa čoho môže návštevník vyhľadávať. Umožňuje tak dynamické rozširovanie vlastností inzerátov podľa ich zaradenia.
- **Message (Správa):** Realizuje komunikačný kanál medzi záujemcom a inzerentom, ale aj systémové notifikácie (`MessageType`). Správy sú logicky zoskupované do konverzácií (vlákien) pomocou odkazu na rodičovskú správu (`parentId`), čo umožňuje prehľadné zobrazenie chatu v používateľskom rozhraní.
- **Favorite (Obľúbené):** Spojovacia entita realizujúca reláciu M:N medzi používateľom a inzerátmi, ktoré si uložil na neskoršie prehliadnutie.
- **Report (Nahlásenie):** Slúži na moderáciu obsahu. Uchováva informáciu o sťažnosti na konkrétny inzerát, dôvod nahlásenia (`ReportReason` – napr. `SPAM`, `SCAM`, `FAKE`) a stav riešenia administrátorom (`ReportStatus`).
- **Payment (Platba):** Zaznamenáva finančné transakcie medzi záujemcom (`renter`) a vlastníkom inzerátu (`owner`), pričom sleduje stav platby (`PaymentStatus`) a obdobie prenájmu/poskytnutia služby.

- **StaticPage a BlogPost (CMS obsah):** Slúžia na správu obsahu priamo z admin panelu. Obsahujú HTML text, SEO metadáta a stav publikovania (`DRAFT` / `PUBLISHED`), čím umožňujú administrátorom upravovať informačné podstránky a blog bez zásahu do kódu.
- **ClickEvent (Analytika):** Špeciálna entita určená na zber údajov o správaní používateľov. Zaznamenáva typ interakcie (`eventType`), cieľový inzerát alebo kategóriu a segmentačné údaje (pohlavie, typ účtu), čo slúži ako podklad pre administratívne štatistiky a grafy.

### 3.4.2 Entitno-relačný diagram (ERD)

ER diagram vizualizuje prepojenia medzi týmito entitami.

- Relácia medzi `User` a `Advertisement` je typu **1:N**, keďže jeden používateľ môže vytvoriť viacero inzerátov, ale inzerát patrí vždy jednému autorovi.
- Podobne `Category` a `Advertisement` tvoria vzťah **1:N**, kde každá kategória zastrešuje množinu prislúchajúcich ponúk.
- Entita `Category` má definovanú **unárnu reláciu 1:N** sama na seba (cez `parentId`), čím sa vytvára stromová štruktúra (hlavné kategórie a podkategórie).
- Entita `Favorite` predstavuje klasickú dekompozíciu vzťahu **M:N** medzi používateľom a inzerátom.
- Entita `Payment` prepája inzerát s dvoma používateľmi (vlastníkom a platcom), čo vytvára komplexnejšiu sieť vzťahov nad tabuľkou používateľov.

## 3.5 Návrh REST API rozhrania a autentifikačného mechanizmu (JWT)

Komunikácia v systéme prebieha prostredníctvom REST API (Representational State Transfer), ktoré využíva protokol HTTP a formát JSON na výmenu údajov. API je navrhnuté ako bezstavové, čo znamená, že každá požiadavka od klienta musí obsahovať všetky informácie potrebné na jej spracovanie, vrátane overenia identity.

### 3.5.1 Návrh endpointov a smerovania

API je logicky rozdelené do modulov, ktoré zodpovedajú doménam v databáze. Každý modul definuje sadu koncových bodov (endpointov) pre operácie CRUD (Create, Read, Update, Delete) a špecifickú biznis logiku.

| Modul | Základná cesta | Významné operácie |
|---|---|---|
| **Auth** | `/auth` | Registrácia, prihlásenie, získanie profilu prihláseného používateľa. |
| **Advertisements** | `/advertisements` | Vyhľadávanie, tvorba inzerátov, schvaľovanie (admin), mapa. |
| **Messages** | `/messages` | Odosielanie dopytov, správa konverzácií a chat. |
| **Analytics** | `/analytics` | Zaznamenávanie kliknutí ( `/click` ) a generovanie reportov. |

**Tabuľka 1:** Prehľad hlavných modulov API a základných ciest

Všetky endpointy sú zdokumentované pomocou nástroja Swagger, ktorý je prístupný na adrese `GET /api/docs`. Táto dokumentácia slúži ako kontrakt medzi backendom a frontendovými tímami (v tomto prípade aplikáciami `platform`, `user` a `admin`).

### 3.5.2 Autentifikačný mechanizmus (JWT)

Na zabezpečenie prístupu k chráneným zdrojom využíva platforma JSON Web Token (JWT). Tento mechanizmus umožňuje overiť identitu používateľa bez nutnosti ukladať informácie o relácii (session) na serveri.

**Proces autentifikácie a autorizácie:**

1. **Prihlásenie:** Používateľ odošle prihlasovacie údaje na `/auth/login`. Server overí heslo (pomocou `bcrypt` s 10 salt rounds) a ak je správne, vygeneruje podpísaný JWT token.
2. **Payload tokenu:** Token obsahuje zakódované informácie ako `userId`, `email` a `role` ( `ADMIN` alebo `USER` ).
3. **Autorizované požiadavky:** Klient ukladá token a posiela ho v hlavičke každého ďalšieho volania: `Authorization: Bearer <token>`.
4. **Validácia na serveri:** API využíva `JwtAuth Guard` na overenie podpisu tokenu a `Roles Guard` na kontrolu, či má používateľ dostatočné oprávnenia (napr. prístup do admin panelu).

### 3.5.3 Validácia a transformácia údajov

Pre zabezpečenie integrity systému je každá prichádzajúca požiadavka validovaná pomocou DTO (Data Transfer Objects). Využitie globálneho `ValidationPipe` v `NestJS` zabezpečuje:

- **Whitelist:** Odstránenie polí, ktoré nie sú definované v DTO, čím sa bráni neoprávneným zmenám v databáze.
- **Transformácia:** Automatický prevod reťazcov z URL parametrov na číselné typy alebo objekty, čo zjednodušuje prácu v službách (services).

### 3.5.4 Spracovanie chýb (Error Handling)

API využíva štandardizované HTTP status kódy (napr. `400 Bad Request` pre chyby validácie, `401 Unauthorized` pre neautorizovaný prístup, `403 Forbidden` pre nedostatočné práva a `404 Not Found` pre nenájdené zdroje). Systém vracia jednotný formát chybových hlásení vo formáte JSON, čo umožňuje ich ľahšie a konzistentné spracovanie na strane klientskych aplikácií (frontendu).
Tento návrh API a bezpečnosti zaručuje, že platforma je robustná, odolná voči bežným útokom a pripravená na komunikáciu s rôznymi typmi klientskych zariadení.

# 4 Metodika a použité technológie

Pri vývoji komplexnej inzertnej platformy som zvolil prístup, ktorý kladie dôraz na rýchlosť vývoja, bezpečnosť kódu a jednoduchú údržbu. Celý ekosystém je postavený na moderných JavaScriptových technológiách, ktoré umožňujú zdieľanie logiky medzi serverovou a klientskou časťou.

## 4.1 Full-stack vývoj v jazyku TypeScript

Základným stavebným kameňom celého projektu je jazyk *TypeScript*. Ide o nadstavbu JavaScriptu, ktorá do dynamického sveta webového vývoja prináša statické typovanie. Pre projekt tohto rozsahu, kde komunikujú tri frontendové aplikácie s jedným API, je použitie TypeScriptu kľúčovým rozhodnutím z niekoľkých dôvodov:

### 4.1.1 Typová bezpečnosť a eliminácia chýb

TypeScript umožňuje definovať presné štruktúry dát (rozhrania a typy) už počas písania kódu. Vďaka tomu dokáže vývojové prostredie (IDE) okamžite upozorniť na chyby, ktoré by sa v čistom JavaScripte prejavili až počas behu aplikácie (napr. prístup k neexistujúcemu poľu objektu). V projekte sú takto ošetrené všetky kľúčové entity, ako sú inzeráty, používatelia či správy. Zatiaľ čo TypeScript zabezpečuje internú konzistenciu kódu počas vývoja (*compile-time*), v kombinácii s knižnicou `class-validator` na backende garantuje, že do systému nevstúpia nevalidné dáta ani z externých HTTP požiadaviek (*runtime*).

### 4.1.2 Zdieľanie kódu v monorepe

Využitie TypeScriptu v prostredí monorepa mi dovoľuje vytvoriť balík `packages/shared`, kde sú definované DTO (*Data Transfer Objects*). Tieto objekty slúžia ako záväzný kontrakt medzi backendom a frontendom.

Keď v `api` zmením štruktúru odpovede pre detail inzerátu, TypeScript ma v aplikáciách `platform` alebo `admin` automaticky upozorní na všetkých miestach, kde je potrebné kód aktualizovať.

Tento mechanizmus zabezpečuje *end-to-end* typovú bezpečnosť, čo dramaticky zvyšuje stabilitu systému pri zmenách v databázovom modeli.

*Príklad zdieľaného DTO v balíku `packages/shared` (využitie tried pre runtime validáciu):*

```
import {
  IsString, IsOptional, IsNumber,
} from 'class-validator';

export class CreateAdvertisementDto {
  @IsString()
  title: string;

  @IsString()
  description: string;

  @IsOptional()
  @IsNumber()
  price?: number;

  // ...
}
```

*Poznámka: V NestJS sa pre DTO odporúča používať triedy (`class`) namiesto rozhraní (`interface`). Dôvodom je, že rozhrania v JavaScripte po kompilácii úplne zmiznú, kým triedy zostávajú zachované aj v* runtime.

*To umožňuje knižniciam ako `class-validator` alebo Swagger čítať metadáta o typoch polí.*

### 4.1.3 Lepšia čitateľnosť, dokumentácia a Developer Experience (DX)

Kód napísaný v TypeScripte je v podstate „samodokumentovateľný“. Akýkoľvek iný vývojár (alebo administrátor systému) dokáže z definícií typov okamžite pochopiť, aké dáta funkcia prijíma a čo vracajú jednotlivé API endpointy. To je dôležité najmä pri modulárnej architektúre NestJS, kde sú jednotlivé služby a kontroléry jasne definované pomocou dekorátorov a typov.

Zároveň TypeScript výrazne zlepšuje *Developer Experience (DX)* vďaka autokomple-

tizácii (IntelliSense) v moderných IDE (ako je VS Code). Pre vývojára to znamená, že nemusí neustále prepínať okná a pozerať sa do databázovej schémy, pretože mu editor sám našepkáva dostupné polia inzerátu alebo používateľa.

### 4.1.4 Moderné funkcie, dekorátory a kompatibilita

TypeScript mi umožňuje využívať najnovšie funkcie jazyka ECMAScript (ako asynchrónne funkcie, deštrukturalizáciu či voliteľné reťazenie), ktoré sú následne kompilované do optimalizovaného JavaScriptu. To zaručuje, že aplikácia beží efektívne v prostredí Node.js na serveri aj v moderných webových prehliadačoch.

Keďže backend využíva NestJS, naplno sa tu prejavuje sila TypeScriptu aj cez dekorátory (`@Controller`, `@Get`, `@Injectable`). Tie umožňujú reflexiu metadát, čo je pokročilá funkcia, vďaka ktorej NestJS vie, ako poskladať závislosti (*Dependency Injection*).

## 4.2 Backend: Framework NestJS a modulárna architektúra

Backendová časť aplikácie (`apps/api`) je vyvinutá v prostredí NestJS. Ide o progresívny framework pre Node.js, ktorý využíva TypeScript a kombinuje prvky objektovo orientovaného programovania, funkcionálneho programovania a reaktívneho programovania.

### 4.2.1 Modulárna štruktúra a organizácia kódu

Hlavnou výhodou NestJS je jeho striktná modulárna architektúra. Každá logická časť platformy je zapuzdrená do samostatného modulu, ktorý obsahuje vlastné kontroléry, služby a dátové modely.

- **Zapuzdrenie logiky:** Napríklad `AdvertisementsModule` sa stará výhradne o inzeráty, zatiaľ čo `AuthModule` rieši autentifikáciu. Tieto moduly sú navzájom nezávislé a striktne dodržiavajú princíp jednej zodpovednosti (*Single Responsibility Principle* – SRP), čo uľahčuje testovanie a umožňuje pridávanie nových funkcií (napr. platobný modul) bez rizika narušenia existujúceho kódu. Všetky tieto doménové moduly sú následne spojené do jedného hlavného stromu aplikácie prostredníctvom koreňového modulu (`app.module.ts`).

- **Dekorátory a metadáta:** NestJS intenzívne využíva dekorátory (napr. `@Get()`, `@Post()`, `@UseGuards()`), ktoré robia kód čitateľnejším a deklaratívnym. Už na prvý pohľad je jasné, ktorý endpoint je verejný a ktorý vyžaduje administrátorské oprávnenia.

*Príklad použitia dekorátorov v kontroléri (ukážka z `AdvertisementsController`):*

```
@Controller('advertisements')
export class AdvertisementsController {
  constructor(private readonly advertisementsService:
     AdvertisementsService) {}

  @Get('pending/all')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.ADMIN)
  findPending() {
    return this.advertisementsService.findPending();
  }
}
```

V tejto ukážke je jasne vidieť, ako dekorátory `@UseGuards` a `@Roles` zabezpečujú ochranu endpointu tak, aby k nemu mal prístup iba administrátor.

### 4.2.2 Dependency Injection (Injektovanie závislostí)

Framework využíva návrhový vzor *Dependency Injection* (DI), ktorý automatizuje správu inštancií tried.

V praxi to znamená, že ak služba `AdsService` potrebuje pristupovať k databáze cez `PrismaService`, stačí túto závislosť definovať v konštruktore a NestJS sa postará o jej vytvorenie a prepojenie.

Tento prístup zvyšuje modularitu a umožňuje jednoduchú výmenu komponentov, čo je v súlade s princípmi čistého kódu (*Clean Code*).

### 4.2.3 Middleware a Guardy pre bezpečnosť

NestJS poskytuje robustný systém na spracovanie životného cyklu požiadavky (*Request Lifecycle*). Každá prichádzajúca HTTP požiadavka prechádza presne definovanou cestou: najprv sa vykoná *Middleware*, následne *Guards* (strážcovia prístupu), *Interceptors* (zachytávače), *Pipes* (validácia a transformácia dát), až sa nakoniec dostane do samotného kontroléra a služby. Tento jasne štruktúrovaný tok umožňuje oddeliť biznis logiku od infraštruktúrnych a bezpečnostných úloh.

- **Guards (Strážcovia):** Využívam ich na ochranu endpointov. `JwtAuthGuard` overuje platnosť tokenu a `RolesGuard` kontroluje, či má používateľ rolu `ADMIN`.
- **Interceptors a Pipes:** Tieto nástroje slúžia na transformáciu dát (napr. úprava formátu odpovede) a ich validáciu. Vďaka `ValidationPipe` je zaručené, že do systému sa nedostanú nekorektné dáta, ktoré by mohli spôsobiť pád aplikácie alebo bezpečnostnú dieru.

### 4.2.4 Integrácia s Prisma ORM

NestJS sa výborne dopĺňa s nástrojom Prisma. V projekte som vytvoril dedikovaný `PrismaModule`, ktorý spracováva pripojenie k PostgreSQL databáze. Služba `PrismaService` priamo rozširuje triedu `PrismaClient` a využíva zabudované hooky životného cyklu NestJS, konkrétne `onModuleInit`. Vďaka tomu sa spojenie s databázou nadviaže hneď pri štarte aplikácie, čo zaručuje, že API nezačne prijímať požiadavky skôr, než je databáza plne pripravená.

Prisma funguje ako typovo bezpečný most (ORM), ktorý mi umožňuje pracovať s databázou pomocou bežných TypeScript objektov namiesto písania surových SQL dopytov.

Výsledkom je kód, ktorý je menej náchylný na chyby a oveľa rýchlejšie sa vyvíja.

4.3 Frontend: Next.js (App Router), React a Tailwind CSS

4.4 Perzistencia dát: PostgreSQL a Prisma ORM

4.5 Nástroje pre správu kódu a kontajnerizáciu (Git, Docker Compose)

5 Implementácia systému

5.1 Štruktúra monorepa a zdieľanie typov (npm workspaces, packages/shared)

5.2 Realizácia backendových modulov (Auth, Ads, Categories, Search)

5.3 Vývoj klientskych aplikácií

5.3.1 Verejná platforma (vyhľadávanie, mapa, blog)

5.3.2 Používateľská zóna (správa inzerátov a profilu)

5.3.3 Administračný panel (CMS, vizuálny editor GrapesJS, štatistiky)

5.4 Bezpečnostné aspekty: Hashovanie hesiel (bcrypt), validácia vstupov (class-validator) a ochrana účtov (banovanie)

5.5 Systém dynamických filtrov a kategórií

6 Testovanie a výsledky

6.1 Overenie funkčnosti API (Swagger dokumentácia)

6.2 Analytika a monitoring (ClickEvent tracking a segmentácia používateľov)

6.3 Vyhodnotenie používateľského rozhrania a responzivity

7 Diskusia a prínosy práce

7.1 Správa obsahu bez nutnosti redeployu (SiteMenu, SiteConfig)

7.2 Možnosti ďalšieho rozšírenia (platobná brána, externé úložisko médií)



8 Záver